
SMM: A Small Math Model: Siegler’s Developmental Arithmetic Models Recast as an LLM

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Siegler and Shrager’s 1984 strategy choice model (S&S84) anticipated princi-
2 ples now central to Large Language Models (LLMs), including confidence-based
3 retrieval, falling back on backup strategies, and learning from naturalistic data.
4 Our present line of research seeks to recast S&S84 and other efforts in Siegler’s
5 decades-long project, in a modern, neural-network-based architecture analogous
6 to large language models (LLMs). In addition we have extended S&S84 by adding
7 actual counting practice. (whereas S&S84 pre-loaded their prediction matrix
8 with probabilities that were supposed to implicate prior knowledge of counting,
9 but they did not actually train the model with counting examples.) Like LLMs,
10 the SMM inputs symbolic sequences representing counting or addition problems,
11 embeds them, applies an attention mechanism, and produces probabilistic outputs,
12 reproducing hallmark behaviors predicted by strategy choice theory. While per-
13 haps pedagogically useful, our goals are more technical, (a) showing that a single
14 mechanism can account for diverse developmental patterns, and (b) providing a
15 platform for investigating “number common sense” in LLMs. In future work we
16 plan to introduce multiple strategies to test whether the model can capture emer-
17 gent number properties and learn adaptive strategy use.

18 1 Introduction

19 The development of mathematical cognition has long been understood through the lens of strat-
20 egy choice theory (e.g., (Siegler and Shrager, 1984)). Children use multiple strategies—counting,
21 retrieval, decomposition—and select among them probabilistically based on associative strength be-
22 tween problems and likely answers, confidence criteria, and performance experience with various
23 strategies. The Siegler framework emphasizes mechanisms that align well with the probabilistic
24 underpinnings, uncertainty estimation, and fallback mechanisms of modern AI. Large Language
25 Models (LLMs) have revolutionized natural language processing by scaling these same principles,
26 producing probabilistic outputs, estimating confidence, and, in some cases, dynamically selecting
27 backup strategies. Moreover, Siegler et al trained their models on realistically distributed problems,
28 similarly to way that modern LLMs are trained on naturally occurring language.

29 Our Small Math Model (SMM) is an attempt to revisit Siegler’s long term project in a contempo-
30 rary architecture. We have two hypotheses: First, that we can unify all of the models that comprise
31 Siegler’s early math learning project in an architecture that provides a natural explanation for seem-
32 ingly diverse phenomena, including learning to count, learning to add and other arithmetic, learning
33 to use strategies in appropriate settings, and learning new strategies. Our second hypothesis is that
34 the SMM can lead to improved number commonsense in LLMs, something that LLMs have been
35 widely noted as lacking (e.g., [need a citation]).

2 Architecture

The SMM processes input sentences of the form “a operator b” where $a, b \in \{0, \dots, 9\}$ and operator $\in \{+, \dots\}$. The symbolic numerical arguments are “embedded” – mapped to a dense vector in a continuous space, enabling the model to capture emergent relationships among numbers (for example, even versus odd, relative magnitude, or common configurations such as base-five hand groupings). These embeddings feed into a recurrent processing core that implements a mixture of symbolic strategies, including finger counting and direct addition.¹ A learned gating (‘quasi-attention’, QA) mechanism modulates how the model uses the operands, conditioning on the operator so that it emphasizes different input features in different contexts (counting vs. addition). Through training, the QA mechanism learns patterns of attention that support distinct strategies: faced with a counting problem, it learns to amplify the representation of the current number in a sequence, facilitating stepwise incrementing, whereas faced with an addition problem it learns to highlight both operands simultaneously, capturing their relational structure.

Like S&S84, our SMM incorporates an explicit *finger counting* auxiliary strategy. Answer confidence is estimated from the entropy of the output distribution. When confidence falls below a threshold, the model defers to an external finger-counting procedure that generates the correct result step by step. The output of this routine is then fed back into training, analogous to the way LLMs can be augmented with external tools or chains-of-thought and subsequently distilled back into the base model.

Counting is trained from the very beginning of each run, while addition can be introduced at different time points to investigate how prior knowledge influences subsequent learning. After each problem, strategy-specific confidence values are updated according to correctness, and these values govern stochastic selection of strategies on subsequent trials. This architecture therefore supports mechanistic analysis of how new procedures compete with, reinforce, or interfere with existing ones in the course of skill acquisition.

Training follows a curriculum schedule in which problem difficulty and task type (counting vs. addition) are varied according to Gaussian flows. This is directly inspired by curriculum learning practices in LLM training, where data distributions are staged over time to improve stability and generalization. Learning rates and confidence thresholds are annealed across steps, further paralleling hyperparameter scheduling in large-scale models.

The code for the SMM is available here under and MIT Open Source liscence: <https://github.com/jeffshrager/smm>

3 Training

Curriculum Learning. Training follows a Gaussian curriculum schedule that gradually increases task difficulty. Early in training, simpler problems (e.g., sums with smaller operands) dominate; over time, harder problems appear with greater probability. This smooth progression mirrors natural learning trajectories while avoiding abrupt jumps.

Logging and Evaluation. The implementation includes structured logging of accuracy, entropy, and error distributions across problem types. These diagnostics allow comparison to known developmental error patterns, such as over-reliance on counting or failure to generalize across contexts.

4 Results

The figures depict sweeping the point at which the amount of addition training becomes significant. Figure ?? shows both constructive and destructive interference; Prior experience with counting can either scaffold or hinder the acquisition of addition, depending upon the timing of addition’s introduction. When addition is introduced too early (e.g., 2000 steps), correctness drops although it recovers rapidly with more addition and counting experience. Figure ?? shows how the use of finger counting rises rapidly, and then just as rapidly drops out once confidence in the answer comes

¹While the network itself is a feedforward model, the training regime and task structure make it functionally recurrent: its own outputs are fed back as inputs across steps to implement counting and addition.

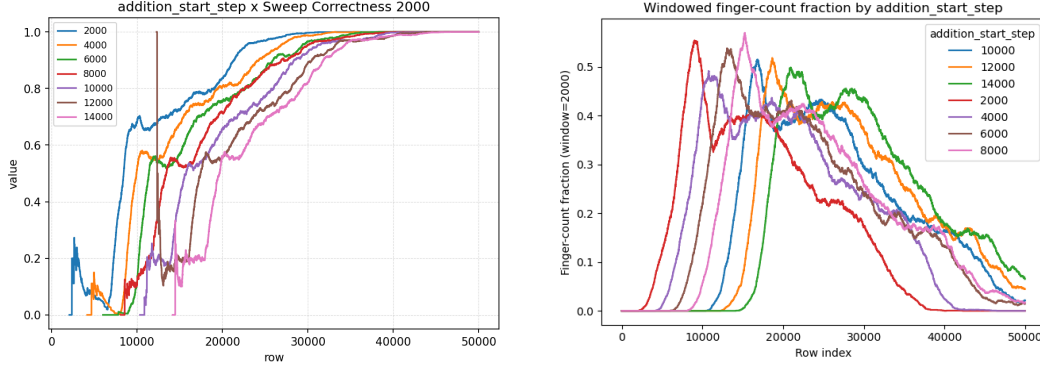


Figure 1: Addition correctness (left) and use of finger counting (right) as a function of training step, for different addition-start times. Counting is always trained from the beginning. Destructive interference takes place near the beginning of practice if addition is introduced too soon, although this it rapidly overcome by addition training.

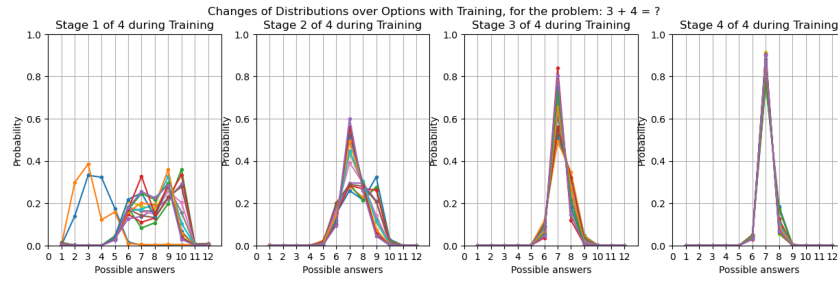


Figure 2

above the confidence threshold. This change in addition answer confidence is depicted in figure 2; Note that early on the possible answers are biased to incorrect results due to addend-bias (for the problem $3+4$ tend to say 3 or 4), counting bias (tend to say 5, from the counting sequence $3 \rightarrow 4 \rightarrow 5$), and errors in finger counting due to poor counting training (this many answers around, but not at 7, i.e., 6,8,9,...). Notice as well that confidence is distributed among these various answers. Yet, with experience these biases fall off, and the correct answer (7) becomes nearly the only likely answer with nearly 1.0 confidence.

5 Discussion and Next Steps

Our SMM modernizes and extends earlier work by Shrager et al. by explicitly drawing analogies to contemporary LLM architectures: embeddings, gating/attention, cross-entropy loss, curriculum learning, and tool use. This vastly simplified architecture provides a testbed for studying how counting priors, confidence estimation, and quasi-attention interact in a controlled setting, with the ultimate goal of bridging symbolic and neural perspectives on mathematical reasoning.

These findings extend Siegler and Shrager’s framework into a computational setting, demonstrating how adaptive strategy competition can give rise to nontrivial transfer effects. The embedding-based architecture further allows us to investigate whether number embeddings capture meaningful latent properties (e.g., even/odd or base-five hand configurations), which we leave to future work.

The SMM serves both as a modern continuation of Siegler’s project and as a testbed for probing fundamental issues in AI reasoning. Our immediate next steps involve expanding beyond a single strategy, allowing the model to learn when to apply different strategies, and incorporating number embeddings to test whether they can capture emergent numerical properties and support improved strategy selection.

The SMM illustrates how modern neural implementations can instantiate and extend classical cognitive theories. By reframing Siegler and Shrager’s ideas in computational terms aligned with LLMs, we obtain a platform for mechanistic analysis of symbolic reasoning in tractable domains, opportunities to probe representations directly (e.g., via similarity analysis or attention visualization), and a bridge between cognitive science and AI, highlighting shared principles of probabilistic selection, confidence estimation, and adaptive strategy use.

Although the SMM may have pedagogical utility, its main contributions are twofold: (a) it brings the Siegler modeling program into the LLM era, demonstrating that a single mechanism can reproduce diverse developmental patterns within a contemporary architecture; and (b) it provides a platform for investigating number common sense in systems modeled after LLMs, an ability that remains fragile in current large-scale AI.

Looking forward, the most important step is to move beyond a single strategy. We plan to implement multiple strategies and allow the model to learn which are appropriate for which types of problems. This expansion will allow closer alignment with empirical findings on children’s adaptive strategy choice, while testing whether embeddings can capture properties of numbers (e.g., parity, magnitude relations, proximity to 5 or 10) that influence strategy efficiency.

While the current implementation uses one-hot encodings, we will introduce **learned embeddings** for numbers and operators. Embeddings should enable the discovery of emergent number properties and support strategy-specific generalizations. To test this, we will design pseudo-strategies with error profiles dependent on number features—for instance, a “finger counting” strategy that excels at +1 problems, or a +5 strategy that works best when an operand is 5. By varying error proneness we can examine whether embeddings capture such latent features. Probes (e.g., linear classifiers on embeddings) and behavioral splits (e.g., 1+ problems vs. 5+ problems) will serve as evaluation tools.

An important implication of these results is the potential value of **phased training** in both human and machine learning. In education, this is sometimes called *progressive deepening* (e.g., Fayek et al. (2020), Hacoen and Weinshall (2019)), and in cognitive science it echoes Elman et al.’s (1996 Elman et al. (1996)) “starting small” hypothesis. Our model shows that first learning to count, and only later to add, can yield constructive rather than destructive interference. By analogy, staged curricula in large language models may help establish core abilities—such as number sense—before introducing higher-level reasoning, supporting more robust common sense in numerical domains and perhaps beyond.

Together, these extensions will provide a richer testbed for exploring how modern neural systems can internalize structured numerical knowledge, reconcile competing strategies, and develop more robust common sense. In doing so, the SMM continues Siegler’s tradition of mechanistic modeling while directly engaging with central challenges in today’s AI research.

References

- Elman, J. L., Bates, E. A., Johnson, M. H., Karmiloff-Smith, A., Parisi, D., and Plunkett, K. (1996). *Rethinking Innateness: A Connectionist Perspective on Development*. MIT Press, Cambridge, MA.
- Fayek, H. M., Srivastava, N., and Jin, J. (2020). Progressive learning: A deep learning framework for continual learning. *Frontiers in Artificial Intelligence*, 3:7.
- Hacoen, G. and Weinshall, D. (2019). On the power of curriculum learning in training deep networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *ICML ’19*, pages 2535–2544. PMLR.
- Siegler, R. S. and Shrager, J. (1984). Strategy choices in addition and subtraction: How do children know what to do? In *Origins of Cognitive Skills*. Erlbaum.